

מבוא לתכנות מערכות – מבחן מועד א', סתיו תשע"ז

מרצה: ד"ר איתי שרון

משך המבחן: 180 דקות

חומר עזר מותר: 2 דפים המכילים תאור פקודות ב-C ו-bash. כל חומר עזר אחר אסור.

המבחן כולל 5 שאלות, יש לענות על כולן. בשאלות הדורשות נימוק הקפידו לספק אותו.

יש לענות על טופס הבחינה במקומות המיועדים לכך. ניתן להשתמש במחברת העזר אולם יש להגיש רק את טופס הבחינה.

העמוד האחרון במבחן כולל חומר שיכול (אך לא חייב!) לעזור לכם.

בהצלחה!

המכללה האקדמית תל חי, החוג למדעי המחשב

שאלה 1 (20 נקודות)

א. נתונה הפונקציה אשר אמורה לקרוא שורה מהקובץ שהמצביע אליו נתון ע"י fp ואז להעתיק את השורה שנקראה לתוך dest כך שניתן יהיה להשתמש בה מחוץ לפונקציה. הקוד מכיל מספר שגיאות אותן יש למצוא. מצאו לפחות 5 שגיאות, עבור כל שגיאה כתבו מה היא ומה תהיה התוצאה האפשרית שלה (למשל: שגיאת קומפילציה, דליפת זכרון וכו). מספרי השורות נתונים לנוחיותכם, השתמשו בהם על מנת לתאר את מיקום השגיאות.

הניחו שנעשה include לכל הספריות הנחוצות. הקוד עושה שימוש בפונקציה getline() אותה ראינו בכתה. ניתן למצוא פרטים עליה בדף הפונקציות בסוף המבחן.

```
1 bool readline(FILE* fp, char *dest) {
2     char* buf;
3     size_t bufsize;
4     char* p = dest;
5
6     if(getline(buf, bufsize, fp) <= 0)
7         return false;
8
9     dest = (char*)malloc(sizeof(char)*strlen(buf));
10    while(*(p++) = *(buf++))
11        ;
12    return true;
13 }
```

1. המשתנה dest צריך להתקבל כמצביע למצביע אחרת הזכרון שאליו הוא יצביע יאבד ביציאה מהפונקציה (שגיאת מימוש)

2. המשתנים buf ו-bufsize צריכים להיות מאותחלים לפני ש-ggetline נקראת (שגיאת זמן ריצה)

3. ל-dest לא מוקצה מספיק זכרון – חסר בית אחד בשביל ה-'0' בסוף (שגיאת זמן ריצה)

4. p לא יכול את הכתובת הנכונה שאליה dest אמור להצביע אחרי הקצאת הזכרון (שגיאת זמן ריצה)

5. אין שחרור של buf (דליפת זכרון)

6. המשתנים buf ו-bufsize מועברים כמו שהם ולא ככתובות ל-ggetline (2 שגיאות קומפילציה)

7. אין בדיקה שהקצאת המקום עבור dest הצליחה (שגיאת זמן ריצה אפשרית)

המכללה האקדמית תל חי, החוג למדעי המחשב

ב. ממשו ב-C גרסא של הפקודה tail אשר מקבלת מצביע לקובץ (fp) ומספר שורות (n) ומדפיסה את n השורות האחרונות בקובץ עליו מצביע fp או את כל הקובץ אם מספר השורות המבוקש גבוה יותר ממספר השורות בקובץ.

הניחו ש:

- כל השורה מסתיימת בתו ירידת שורה ('\n')
- המצביע fp הוא חוקי, פעולות הקצאת זכרון או קריאה לפונקציות דוגמת fseek (ראו דף הפונקציות בסוף) מסתיימות בהצלחה

```
void my tail(FILE* fp, int n) {
```

```
    int nlines = 0;
```

```
    char c;
```

```
    while((c=fgetc(fp)) != EOF) {
```

```
        nlines += (c == '\n');
```

```
    }
```

```
    fseek(fp, 0, SEEK SET);
```

```
    while((c=fgetc(fp)) != EOF) {
```

```
        if(nlines <= n)
```

```
            putc(c, stdout);
```

```
        nlines -= (c == '\n');
```

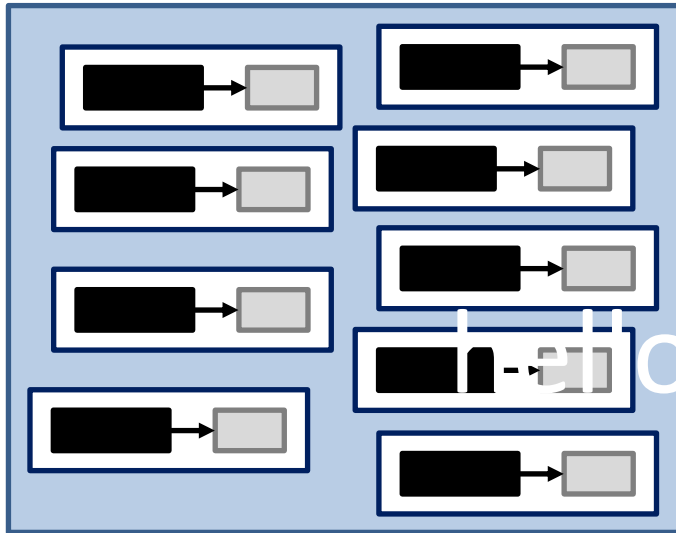
```
    }
```

```
}
```

המכללה האקדמית תל חי, החוג למדעי המחשב

שאלה 2 (30 נקודות)

מבנה הנתונים Map מכיל זוגות של ערכים (values) ומפתחות (keys). הוצאתו של ערך מסויים נעשית על ידי שימוש במפתח שלו. המפתחות במבנה הנתונים Map הם ייחודיים: כל מפתח יכול להופיע בדיוק פעם אחת. הערכים, לעומת זאת, יכולים להופיע מספר פעמים עבור מפתחות שונים. בשאלה זו המפתחות הם תמיד מחרוזות אבל הערכים יכולים להיות מטיפוס כלשהו שאינו ידוע מראש.



התרשים משמאל מתאר Map שבו הערכים הם מספרים שלמים. המפתחות מסומנים בשחור והערכים באפור. כפי שניתן לראות הערך 12 מופיע עבור המפתחות "hello", "chat" ו-"grape".

א. כתבו ממשק (קובץ h) למבנה נתונים Map כמתואר אשר מאפשר את הפעולות הבאות:

- יצירת Map
- הריסת Map
- הוספת ערך עם מפתח שלו ל-Map. הפונקציה צריכה להחזיר bool בהתאם לנכונות הפעולה. הפעולה תמומש ע"י העתקה של המפתח והערך.
- הסרת מפתח והערך שלו על פי המפתח
- החזרת ערך על פי המפתח שלו
- החזרת מספר הזוגות מפתח-ערך במבנה הנתונים
- קבלת מפתח על פי אינדקס. אינדקס חוקי יהיה בין 1 למספר האיברים במבנה הנתונים.

```
#ifndef MAP_H
```

```
#define MAP_H
```

```
typedef struct Map* Map
```

```
Map MapCreate(void* (*cpy)(void*), void (*fre)(void*));
```

```
void MapDestroy(Map m);
```

```
bool MapAdd(Map m, const char* key, void* val);
```

```
void MapRemove(Map m, const char* key);
```

```
void* MapRetrieve(Map m, const char* key);
```

```
int MapSize(Map m);
```

```
const char* MapIterator(Map m, int index);
```

```
#endif
```

המכללה האקדמית תל חי, החוג למדעי המחשב

ב. ה-Map שלנו ימומש ע"י רשימה מקושרת. כתבו את שני ה-structs, עבור ה-Map ועבור איבר ברשימה המקושרת שלו, בהם תעשו שימוש עבור מימוש ה-Map. שלכם. ממשו את פונקציית יצירת ה-Map.

```
typedef struct LNode* LNode;
```

```
struct LNode {  
    const char* key;  
    void* val;  
    LNode next;  
};
```

```
struct Map {  
    LNode head;  
    void* (*cpy element)(void*);  
    void (*fre element)(void*);  
};
```

```
Map MapCreate(void* (*cpy)(void*), void (*fre)(void*)) {  
    Map m = (Map)malloc(sizeof(struct Map));  
    if(m == NULL) {  
        fprintf(stderr, "Failed to allocate memory in MapCreate\n\n");  
        exit(-1);  
    }  
    m->cpy element = cpy;  
    m->fre element = fre;  
    m->head = NULL;  
    return m;  
}
```

המכללה האקדמית תל חי, החוג למדעי המחשב

ג. ממשו את פונקציית הוספת האיבר לקבוצה. הפונקציה תוסיף את הזוג מפתח-ערך ותחזיר true במקרה שבו המפתח לא קיים ב-Map ותחזיר false בלי לשנות את מבנה הנתונים אחרת.

```
bool MapAdd(Map m, const char* key, void* val);
```

```
    LNode n = m->head;
```

```
    while(n != NULL) {
```

```
        if(!strcmp(n->key, key))
```

```
            return false;
```

```
    }
```

```
    n = (LNode) calloc(sizeof(struct LNode), 1);
```

```
    n->key = strdup(key);
```

```
    n->val = m->cpy element(val);
```

```
    n->next = m->head;
```

```
    m->head = n;
```

```
}
```

המכללה האקדמית תל חי, החוג למדעי המחשב

שאלה 3 (15 נקודות)

שאלה זו עוסקת במניפולציות ביטים.

א. ממשו פונקציה אשר מקבלת מספר מטיפוס `unsigned int` ומדפיסה את הייצוג הבינארי שלו למסך.

```
void printBits(unsigned int n) {
```

```
    int i;
```

```
    for(i=8*sizeof(n)-1; i>=0; i--) {
```

```
        putchar((n & (1 << i)) ? '1' : '0', stdout);
```

```
    }
```

```
}
```

המכללה האקדמית תל חי, החוג למדעי המחשב

ב. ממשו פונקציה אשר מקבלת מספר מטיפוס `unsigned int` ומחזירה את המספר האי זוגי הגדול ביותר מטיפוס `unsigned int` אשר אותו ניתן ליצור בעזרת מספר ה-1 ו-0ים שבמספר המקורי.
למשל: אם המספר המקורי הוא 12 (ייצוג בינארי 1100) אז:

- בייצוג הבינארי של המספר אותו תצרו תופיע הספרה 1 פעמיים והספרה 0 תופיע 30 פעמים
- המספר שיווצר יהיה אי זוגי
- לא יהיה מספר אי זוגי אחר אשר בייצוג הבינארי שלו מופיעה הספרה 1 פעמיים והספרה 0 מופיעה 30 פעמים והוא גדול יותר מהמספר אותו החזרתם

```
unsigned int createLargest(unsigned int n) {
```

```
    int i, ones=0, res = 0;
```

```
    for(i=8*sizeof(n)-1; i>=0; i--) {
```

```
        ones += ((n & (1 << i)) != 0);
```

```
    }
```

```
    for(i=0; i<ones-1; i++) {
```

```
        res |= (1 << (31-i));
```

```
    }
```

```
    res |= 1;
```

```
    return res;
```

```
}
```

המכללה האקדמית תל חי, החוג למדעי המחשב

שאלה 4 (20 נקודות)

בשאלה זו תכתבו שתי פונקציות אשר משוות דמיון של טקסט.

ניתן להניח שהקלט בשאלה זו מורכב אך ורק מאותיות בא"ב האנגלי (A-Z – uppercase או a-z – lowercase) ומרווחים.

הגדרות:

- **מילה** מוגדרת כרצף של תווים אשר אינם רווחים באורך 1 או יותר.
- **טקסט** מוגדר כאוסף מילים מופרדות ע"י רווחים. שתי מילים יכולות להיות מופרדות ע"י רווח אחד או יותר, אין משמעות למספר הרווחים בין שתי מילים.
- שתי מילים נקראות **דומות ביחס לפעולת השוואה כלשהי** אם הן מכילות תוים דומים בהתאם לפעולת ההשוואה המוגדרת. למשל: אם נשווה מילים על סמך הערך ה-ascii של התווים מהם הן בנויות אז המילים peace ו-PEACE לא תהיינה דומות בגלל שערכי ה-ascii של אותיות lowercase ו-lowercase הם שונים. לעומת זאת אם נשווה מילים ללא התחשבות בסוג האותיות (uppercase או lowercase) אז המילים peace ו-PEACE תהיינה דומות.
- שני טקסטים נקראים **דומים ביחס לפעולת השוואה כלשהי** אם הם מכילים בדיוק את אותו רצף של מילים דומות אשר ביניהן רווח אחד או יותר. שימו לב שמספר הרווחים יכול להיות שונה בין שני הטקסטים. למשל: שתי השורות הבאות מכילות טקסטים דומים ביחס לפעולת השוואת מילים על סמך הערך ה-ascii של האותיות מהם הן בנויות:

```
Mary had a little lamb
Mary      had      a little      lamb
```

א. כתבו פונקציה לא רקורסיבית וגנרית **ביחס לפעולת ההשוואה בין תווים** אשר משווה שני טקסטים הנתונים על ידי שני מצביעים מטיפוס `const char*`. המצביעים מכילים כתובות של טקסט חוקי המסתיים ב-`'\0'`. הפונקציה תחזיר `true` אם שני הטקסטים דומים או `false` אחרת. הפונקציה צריכה להיות גנרית ביחס לפעולת ההשוואה, כלומר עליה לאפשר השוואה מכל סוג שהוא אשר מוגדר ע"י המשתמש.

```
bool similar(const char* s1, const char* s2, bool (*similar ch)(char a, char b))
{
    while(*s1 || *s2) {
        if(*s1 == '\0')
            s1++;
        else if(*s2 == '\0')
            s2++;
        else if(!(*s1) || !(*s2) || !similar ch(*(s1++), *(s2++)))
            return false;
    }
    return true;
}
```

המכללה האקדמית תל חי, החוג למדעי המחשב

ב. כתבו פונקציה רקורסיבית אשר מקבלת שני מצביעים מטיפוס `const char*` אשר מכילים כתובת של טקסט חוקי המסתיים ב- `'\0'` ומשווה את הטקסט השמור בשני המצביעים. הפונקציה תחזיר `true` אם שני הטקסטים דומים או `false` אחרת. הפונקציה לא צריכה להיות גנרית אלא להתבסס על השוואה פשוטה של תווים. על הפונקציה לעבוד ללא לולאות כלל.

```
bool similar_rec(const char* s1, const char* s2) {  
    if ((*s1 == '\0') && (*s2 == '\0'))  
        return true;  
    if ((*s1 == '\0') || (*s2 == '\0') || (*s1 != *s2))  
        return false;  
    if (*s1 == ' ')  
        return similar_rec(s1+1, s2);  
    if (*s2 == ' ')  
        return similar_rec(s1, s2+1);  
    return similar_rec(s1+1, s2+1);  
}
```

המכללה האקדמית תל חי, החוג למדעי המחשב

שאלה 5 (15 נקודות)

בשאלה זו תכתבו סקריפט bash אשר ימיין קבצים המכילים ספרים לתיקיות בהתאם לכותב הספר ושמו. כל הקבצים המדוברים מסתיימים בסיימת book. ובנויים בפורמט הבא:

- השורה ראשונה מכילה את שם הסופר/ת. השורה מתחילה במילה Author: שלאחריה יש רווח ואחריה, עד סוף השורה, מופיע שם הסופר/ת. ניתן להניח ששם הסופר/ת מכיל רק אותיות בשפה האנגלית ורווחים.
- השורה השנייה מכילה את שם הספר. השורה מתחיל במילה Title: שלאחריה יש רווח ואחריה, עד סוף השורה, מופיע שם הספר. שם הספר יכול להכיל תווים כלשהם.
- החל מהשורה השלישית והלאה מופיע טקסט הספר.

דוגמא לארבע השורות הראשונות של קובץ המכיל את הספר הראשון בסדרת הארי פוטר:

```
Author: JK Rowling
Title: Harry Potter and the Sorcerer's Stone
The boy who lived
Mr. and Mrs. Dursley, of number four, Privet Drive, were proud to say that they
```

א. כתבו את שורות הקוד הבאות:

- שורה המוציאה את שם הסופר/ת מתוך קובץ book. ומחליפה את הרווחים בשם בקו תחתי ("_"). שם הקובץ נתון ע"י המשתנה f, התוצאה תאוחסן במשתנה name. למשל: עבור הדוגמא שלמעלה יכיל המשתנה name את המחרוזת JK_Rowling.

```
name=`head -n1 $f | cut -f2- -d" " | sed 's/ / /g'`
```

- שורה המוציאה את כותרת הספר ומחליפה את כל התווים שאינם מספרים או אותיות קטנות/גדולות בקו תחתי ("_"). שם הקובץ נתון במשתנה f ואת התוצאה יש להכניס למשתנה title. למשל: עבור הכותרת של הספר למעלה יכיל המשתנה title את המחרוזת Harry_Potter_and_the_Sorcerer_s_Stone.

```
title=`head -n2 $f | tail -n1 | cut -f2- | sed 's/[^A-Za-z0-9]/ /'`
```

- שורה המחזירה את מספר המילים בספר ללא שורת שם הסופר ושורת שם הספר. התוצאה תשמר במשתנה nwords.

```
nwords=`egrep -v "^(Author|Title): " | wc -w`
```

המכללה האקדמית תל חי, החוג למדעי המחשב

ב. כתבו סקריפט אשר מקבל שם של תיקייה אשר מכילה קבצי book. הסקריפט יבצע את הפעולות הבאות עבור כל קובץ book::

1. ימצא את שם הסופר ויצור משתנה בשם author בהתאם לסעיף א'1
2. ימצא את כותרת הספר ויצור משתנה בשם title בהתאם לסעיף א'2
3. ימצא את מספר המילים בספר ויצור משתנה nwords בהתאם לסעיף א'3
4. ייצור תיקייה הנושאת את שם הסופר/ת כפי שהוא מופיע במשתנה author תחת התיקייה הנוכחית אם אין תיקייה כזאת עדיין
5. יעתיק את הקובץ הנוכחי לתיקייה מ-(4). במקום השם המקורי ייקרא הקובץ על פי שם הספר בהתאם לתוכן המשתנה title עם סיומת book.
6. ידפיס למסך את תוכן המשתנים author, title, nwords מופרדים ע"י רווחים

למשל: עותק של הספר מהדוגמא למעלה צריך להופיע, לאחר הרצת הסקריפט, תחת

JK_Rowling/Harry_Potter_and_the_Sorcerer_s_Stone.book

בתיקייה הנוכחית.

ניתן להשתמש בשורות הקוד מסעיף א' גם אם לא מימשתם אותן.

ניתן להניח שכל הקלט תקין (קבצים, תיקיית יעד), שהתיקייה הנוכחית מכילה רק את הסקריפט לפני הרצתו אבל תיקיית הקלט יכולה להכיל קבצים נוספים שאינם קבצי book.

```
#!/bin/bash
```

```
if [[ $# -ne 1 ]]; then
```

```
    echo "Usage: $0 <directory>"
```

```
    exit
```

```
fi
```

```
if [[ ! -d $1 ]]; then
```

```
    echo "Error: $1 must be a directory"
```

```
    exit
```

```
fi
```

```
for f in $1/*.book; do
```

```
    author=`head -n1 $f | cut -f2- -d" " | sed 's/ / /g'`
```

```
    title=`head -n2 $f | tail -n1 | cut -f2- | sed 's/[^A-Za-z0-9]/ /'`
```

```
    nwords=`egrep -v ""^(Author|Title): " | wc -w`
```

```
    mkdir -p $author
```

```
    cp $f $author/$title.book
```

```
    echo $author $title $nwords
```

```
done
```

This image shows a single sheet of white paper with horizontal ruling lines. The lines are evenly spaced and run across the width of the page. There are no margins, text, or other markings on the paper.

המכללה האקדמית תל חי, החוג למדעי המחשב

פקודות bash

- cat – מדפיסה תוכן של קובץ למסך
- head – מדפיסה את 10 השורות הראשונות של קובץ למסך (ניתן לשנות את המספר ע"י שימוש ב -n)
- tail – מדפיסה את 10 השורות האחרונות של קובץ למסך (ניתן לשנות את המספר ע"י שימוש ב -n)
- ls – מדפיסה תוכן ספרייה, או רשימה של קבצים אשר מתאימה לתאור שהעביר המשתמש כארגומנט (למשל: ls *.c תדפיס את שמות הקבצים שמסתיימים ב-c. בספרייה הנוכחית)
- sort – ממיינת לקסיקוגרפית קובץ קלט. ניתן למיין מספרית (-n) ולשמור רק עותק אחד של שורות שחוזרות על עצמן (-u)
- wc – מחזירה את מספר השורות (-l), בתים (-c) או מילים (-w) בקובץ
- read – קוראת שורה מה-standard input ומכניסה אותה למשתנה שמועבר ע"י המשתמש כארגומנט
- find – מחזירה רשימה של כל הקבצים אשר מתאימים לתאור המשתמש (ארגומנט -name) בספרייה מסוימת ואלו שנמצאות תחתיה (ארגומנט ראשון לפקודה)
- grep – מחזירה שורות אשר מכילות string מסויים. ניתן להשתמש ב-regular expressions (-e), ולהחזיר שורות שאינן מכילות את ה-string אותו מחפשים (-v)
- egrep – זהה ל-grep -e
- cut – מחזירה שדות מסויימים בכל שורה (-f) בהתאם ל-delimiter (-d). ניתן גם להחזיר בתים מסויימים בכל שורה (-b)
- sed – עורכת טקסט הנתון כקלט. הפעולה s תשנה שורת טקסט.

פעולות על ביטים

- bitwise AND - &
- bitwise OR - |
- bitwise XOR - ^
- right shift - >>
- left shift - <<
- One's complement - ~

פונקציות ב-C

- int putchar(int character, FILE* fp) – כותבת את character ל-fp.
- int fgetc(FILE* fp) – קוראת תו אחד מתוך fp.
- scanf(const char* format,...) – קוראת מידע מתוך stdin בהתאם ל-format, מאחסנת את המידע בכתובות המשתנים שמסופקים. fscanf(FILE* stream, const char* format,...) – אותו דבר כאשר הקלט נקרא מתוך stream.
- sscanf(const char* s, const char* format,...) – אותו הדבר כאשר הקלט נקרא מתוך s.
- int getline(char** lineptr, size_t* n, FILE* stream) – קוראת שורה לתוך lineptr, מקצה זכרון עבור lineptr אם אין מספיק גודל ה-buffer הנוכחי מצוי ב-n). מחזירה את מס' התווים שקראה או -1 אם לא הצליחה לקרוא.
- printf(const char* format,...) – כתיבת מידע ערוך ל-stdout בהתאם ל-format והמשתנים שמסופקים.
- fprintf(FILE* stream, const char* format,...) – אותו דבר כאשר הכתיבה נעשית לתוך stream.
- sprintf(const char* s, const char* format,...) – אותו הדבר כאשר הכתיבה נעשית לתוך s.
- void* malloc(size_t size) – מקצה זכרון בגודל size
- void* calloc(size_t num, size_t size) – מקצה num יחידות בגודל size כל אחת ומאפסת את כל הזכרון שהוקצה.
- void* realloc(void* ptr, size_t size) – מקצה size בתים, מוסיפה/מורידה מכמות הזכרון ש-ptr מצביעה עליו (צריך להיות מוקצה דינמית גם כן), עשויה לשנות את הזכרון המוקצה ואז תעתיק את התוכן של ptr לשטח החדש. מחזירה את כתובת הזכרון שהוקצה.
- const char* strstr(const char* str1, const char* str2) – מחזירה את המקום שבו מופיעה str2 ב-str1 (או NULL אם היא לא מופיעה)

המכללה האקדמית תל חי, החוג למדעי המחשב

- `int strcmp(const char* str1, const char* str2)` – משווה את str1 ל-str2, מחזירה 0 אם הן זהות, ערך קטן מ-0 אם str1 קטן לקסיקוגרפית מ-str2 או ערך גדול מ-0 אחרת.
 - `size_t strlen(const char* str)` – מחזירה אורך של מחרוזת.
 - `int fseek(FILE *fptr, long offset, int origin)` - מזיזה את fptr ל-offset בתיים ביחס ל-origin. המשתנה origin יכול להיות
 - `SEEK_SET` – תחילת הקובץ
 - `SEEK_CUR` – מיקום נוכחי
 - `SEEK_END` – סוף הקובץ
- הפונקציה תחזיר 0 בהצלחה או ערך שונה מ-0 אחרת.